

Dictionaries

Christopher Martin - *Bowdoin College* 
c.martin@bowdoin.edu 

Creating a Dictionary

Dictionaries can be created empty or with initial data using `{}`

Pairs in a dictionary are formed by separating the key and value with a colon

For example: `key: value`

Where `key` and `value` can be anything! Even other data structures!

```
1 empty: dict = {}
2 has_initial_data: dict[str, int] = {"A": 9, "B": 3, "C": 1}
3
4 print(f"The empty dict: {empty}")
5 print(f"The dict with data: {has_initial_data}")
```

The empty dict: {}

Creating a Dictionary

To make longer pairs more easily visible, we will sometimes split up the initialization of the dictionary across multiple lines like so:

```
1 has_initial_data: dict[str, int] = {  
2     "A": 9,  
3     "B": 3,  
4     "C": 1  
5 }  
6  
7 print(f"The dict with data: {has_initial_data}")
```

The dict with data: {'A': 9, 'B': 3, 'C': 1}

Note that this doesn't change the way it is stored/printed!

Essential Dictionary Methods

The following methods do most of the basic operations we need!

Method	Description
<code>my_dict.get(k)</code>	Returns the value associated with the key <code>k</code> if it exists in <code>my_dict</code> , returns <code>None</code> or an optionally specified default value otherwise
<code>my_dict.keys()</code>	Returns a list containing all of the keys from each pair in <code>my_dict</code>
<code>my_dict.values()</code>	Returns a list containing all of the values from each pair in <code>my_dict</code>
<code>my_dict.items()</code>	Returns a list containing all of the pairs in <code>my_dict</code>
<code>my_dict.pop(k)</code>	Removes the pair with the specified key <code>k</code> from <code>my_dict</code> , returns the removed value associated with the key <code>k</code> but causes an error if <code>k</code> is not in <code>my_dict</code>

What essential behaviors are missing?



Adding and Modifying pairs

Adding and modifying pairs in a dictionary are done using the `[]` !

If the specified key isn't already in the dictionary, you add it with the provided value; otherwise, you change its value!

```
1 grades: dict[str, int] = {"A": 9, "B": 3, "C": 1}
2
3 print("Before:", grades)
4
5 # The key "D" isn't already in grades so this adds the pair "D": 2
6 grades["D"] = 2
7
8 # The key "A" is already in grades so this changes the pair "A": 9 to "A": 7
9 grades["A"] = 7
10
11 # You can also modify a pair's value like any other variable!
12 grades["B"] += 2
13
14 print("After: ", grades)
```

Before: {'A': 9, 'B': 3, 'C': 1}

After: {'A': 7, 'B': 5, 'C': 1, 'D': 2}

Looping through Dictionaries

We can customize which aspects of the dictionary we loop through!

```
1 grades: dict[str, int] = {"A": 9, "B": 3, "C": 1}
2
3 print("for key in grades", end=": ")
4 for key in grades: # Implicitly calls .keys()
5     print(key, end=" ")
6 print("\n")
7
8 print("for key in grades.keys()", end=": ")
9 for key in grades.keys():
10     print(key, end=" ")
11 print("\n")
12
13 print("for value in grades.values()", end=": ")
14 for value in grades.values():
15     print(value, end=" ")
```

for key in grades: A B C

for key in grades.keys(): A B C

for value in grades.values(): 9 3 1

Looping through Dictionaries

The `items()` method allows us to access the pairs themselves!

```
1 grades: dict[str, int] = {"A": 9, "B": 3, "C": 1}
2
3 print("for item in grades.items()", end=": ")
4 for item in grades.items():
5     # 'item' is a tuple so you can access the key at item[0] and the value at item[1]
6     print(item, end=" ")
7 print("\n")
8
9 print("for key, value in grades.items():")
10 for key, value in grades.items(): # You can 'destructure' the item tuple!
11     print(f"Key: {key}, Value: {value}") # This makes printing super customizable!
```

```
for item in grades.items(): ('A', 9) ('B', 3) ('C', 1)
for key, value in grades.items():
Key: A, Value: 9
Key: B, Value: 3
```

The `in` Operator

Using the `in` operator on a dictionary only checks their keys!

```
1 grades: dict[str, int] = {"A": 9, "B": 3, "C": 1}
2
3 print("The grade A is in the dictionary:", "A" in grades)
4 print("The number 1 is in the dictionary:", 1 in grades)
5
6 print("\n")
7
8 # We can use in on the list of values!
9 print("The number 1 is in the dictionary:", 1 in grades.values())
```

```
The grade A is in the dictionary: True
The number 1 is in the dictionary: False
The number 1 is in the dictionary: True
```

Accessing with `[]` vs `get`

Accessing with the `[]` operator

`[]` allows us to get the value associated with a key if it exists!
Fails if the key doesn't exist!

```
1 grades: dict[str, int] = {  
2     "A": 9,  
3     "B": 3,  
4     "C": 1  
5 }  
6  
7 print("Number of A's:", grades["A"]) # ✓ Works!  
8 print("Number of E's:", grades["E"]) # ✗ Crashes!
```

```
Traceback (most recent call last):  
  File "/app/output.s", line 8, in <module>  
    print("Number of E's:", grades["E"]) # ✗  
Crashes!  
~~~~~  
KeyError: 'E'
```

Accessing with the `get` method

`get()` allows us to get the value associated with a key if it exists!
Returns `None` if the key doesn't exist!

```
1 grades: dict[str, int] = {  
2     "A": 9,  
3     "B": 3,  
4     "C": 1  
5 }  
6  
7 # With no default value  
8 print("Number of E's:", grades.get("E"))  
9  
10 # With a default value of 0  
11 print("Number of E's:", grades.get("E", 0))
```

```
Number of E's: None  
Number of E's: 0
```